

Indirect Prompt Injection과 Lethal Trifecta

에이전트형 AI를 안전하게 설계하는 법

비신뢰 입력 처리, 민감 데이터 접근, 외부 전송·상태변경 액션이
한 세션과 한 경로에 동시에 결합되면 구조적으로 취약해집니다.

핵심 메시지: 더 강한 프롬프트보다 경계와 권한 분리가 먼저입니다.

왜 에이전트형 AI는 기존 앱보다 더 위험한가

하나의 자연어 루프에 너무 많은 권한이 연결됨

- 브라우징, 이메일, 문서, DB, API, MCP를 한 세션에서 호출합니다.
- 사용자 요청과 외부 콘텐츠가 같은 컨텍스트 안에서 섞입니다.
- 모델이 계획과 실행을 동시에 담당하면 판단 경계가 흐려집니다.

공격자는 "코드"가 아니라 "콘텐츠"를 넣어도 됨

- 웹 페이지, 이메일, 문서, 티켓, PDF 안의 문장이 행동을 바꿀 수 있습니다.
- 이 입력은 종종 로그·검색·요약 단계를 거쳐 더 넓게 전파됩니다.
- 그래서 문제는 모델 성능보다 시스템 결합 구조에 가깝습니다.

Indirect Prompt Injection이란 무엇인가

정의

- 사용자가 직접 입력하지 않은 외부 콘텐츠의 지시가 에이전트 컨텍스트에 들어와 계획이나 행동을 바꾸는 공격입니다.
- 입력 표면은 웹 페이지, 이메일 본문, 첨부 문서, 이슈 티켓, 위키 문서, PDF, MCP 응답까지 포함됩니다.
- 전통적 인젝션과 달리, 악성 문자열이 "사람이 읽는 콘텐츠"처럼 보인다는 점이 핵심입니다.

?? ? -> ???? ? ???? ? -> ? ? ? ???? ? -> ?? ? ???? ? -> ?? ? ? ? ? ? ? ?

Lethal Trifecta: 세 조건이 동시에 모이면 위험해진다

1. 비신뢰 입력

웹, 이메일, 문서, 티켓, 검색 결과, 외부 MCP 응답처럼 신뢰할 수 없는 입력

2. 민감 데이터 접근

내부 문서, 메일함, 비공개 노트, 토큰, DB, 파일시스템 같은 민감 데이터 접근 권한

3. 외부 액션

외부 전송, 메시지 발송, 상태 변경, 시스템 명령 실행, 티켓 수정, 결제 요청

중양 위험

세 조건이 한 경로에서 합쳐지면 데이터 유출과 비인가 액션이 "모델이 속는 순간" 바로 실현될 수 있습니다.

공격 흐름 예시

1. 공격자가 악성 이메일이나 문서를 시스템 안으로 유입시킵니다.
2. 사용자는 "이 메일 요약하고 관련 문서 찾아줘" 같은 정상 요청을 보냅니다.
3. 에이전트는 메일 본문을 읽고 내부 검색 도구로 추가 문서를 가져옵니다.
4. 악성 지시가 모델의 우선순위를 바꾸어 secret, token, 내부 링크를 요약에 포함시킵니다.
5. 후속 단계에서 외부 메일, Slack, webhook, 브라우저 요청으로 데이터가 유출됩니다.

왜 "더 강한 프롬프트"만으로는 못 막는가

프롬프트는 정책 선언이지 강제 경계가 아님

- 모델은 우선순위를 추론합니다. 선언한 규칙이 항상 이기지 않습니다.
- 긴 context와 다단계 작업에서는 지시 충돌과 오해가 누적됩니다.
- 악성 문장이 일반 업무 텍스트처럼 보이면 분리 자체가 어렵습니다.

근본 방어는 피해를 제한하는 구조

- 속더라도 민감 데이터와 외부 액션이 바로 연결되지 않게 해야 합니다.
- 도구 권한, 컨텍스트 범위, 승인 단계, 외부 전송 경로를 시스템이 강제해야 합니다.
- 즉 핵심은 탐지 자체보다 피해 범위를 제한하는 구조입니다.

Rule of Two: 원칙 자체는 단순하다

META AGENTS RULE OF TWO

- 한 세션, 한 실행 경로, 한 자동화 단계에서 아래 세 가지를 동시에 주지 않습니다.

비신뢰 입력 처리

외부 콘텐츠를 읽고 해석하는 능력

민감 데이터 접근

내부 비공개 정보에 접근하는 능력

외부 전송·상태변경

외부로 보내거나 시스템 상태를 바꾸는 능력

설계 목표는 "셋 중 최대 둘까지만 결합"입니다. 나머지 하나는 다른 단계나 다른 승인 경계 뒤로 밀어냅니다.

Rule of Two: 설계에 적용하면 어떻게 달라지나

문서 요약 전용 에이전트

비신뢰 입력 + 제한된 읽기

- 외부 전송과 상태변경이 없으면 피해 범위가 크게 줄어듭니다.

승인된 보고서 발송 에이전트

정제된 내부 데이터 + 외부 전송

- 비신뢰 입력을 직접 읽지 않으면 주입 표면을 낮출 수 있습니다.

코드 수정 에이전트

비신뢰 입력 + 상태변경

- 민감 데이터 저장소 접근을 끊으면 유출 경로를 줄일 수 있습니다.

관리자 에이전트

민감 데이터 + 상태변경

- 외부 콘텐츠 유입을 끊고 승인 단계를 강제해야 안전합니다.

핵심은 한 에이전트가 모든 역할을 다 하지 않게 나누는 것입니다.

대표 구조: CaMeL 계열 접근

계획을 세우는 모델

- 사용자 요청을 읽고 실행 계획, 제한된 코드, 도구 호출 초안을 만듭니다.
- 외부 문서 원문을 그대로 읽지 않도록 범위를 좁힙니다.
- 권한 모델과 실행 가능한 액션 집합을 명시적으로 받습니다.

외부 콘텐츠를 분리하는 모델

- 외부 콘텐츠는 schema 기반 필드 추출이나 요약으로만 전달합니다.
- 명령형 문장은 제거하고, 필요한 사실만 구조화된 값으로 넘깁니다.
- 계획 모델은 정제 결과만 소비하고 원문과 섞이지 않습니다.

??? ?? -> ?? ?? ?? ??? -> ?? ?? -> ??? ? ? ? ? -> ??? ??? -> ?? ?? ?? ??

방어 패턴 1: Dual LLM 또는 입력 경계 분리

어떻게 나누나

- Planner는 trusted user request만 읽습니다.
- Extractor는 untrusted content만 읽고 구조화 결과만 냅니다.
- 두 경로의 프롬프트, 도구, 세션 메모리를 분리합니다.

보호 효과

- 악성 문장이 계획 수립 루프로 직접 들어오지 않습니다.
- 계획 모델이 읽는 것은 필드화된 데이터라 공격 표현력이 줄어듭니다.
- 로그와 감사 관점에서도 "누가 무엇을 읽었는지"가 선명해집니다.

실무 포인트: 추출 모델 출력은 JSON 스키마, enum, 최대 길이 제한처럼 해석 범위를 좁히는 편이 좋습니다.

방어 패턴 2: Plan-Then-Execute

실행 전에 계획을 고정

- 먼저 "무슨 데이터를 읽고 어떤 도구를 쓸지" 계획을 고정합니다.
- 그다음 별도 executor가 허용된 step만 실행합니다.
- 중간에 외부 콘텐츠가 새 지시를 넣어도 계획을 다시 쓰지 못하게 합니다.

어디에 유리한가

- 브라우징 후 이메일 발송, 문서 검색 후 티켓 수정 같은 다단계 작업
- 승인 가능한 중간 산출물과 audit log가 필요한 조직 환경
- 도구 호출을 허용 목록 기반으로 강제해야 하는 경우

?? ?? -> ?? ?? -> ??? ?? -> ?? ??

방어 패턴 3: Code-Then-Execute

자연어를 바로 실행하지 않고 제한된 표현으로 바꾼다

- 계획 모델이 자유 서술 대신 제한된 DSL, typed action list, restricted code를 생성합니다.
- executor는 허용된 명령 집합만 해석하고, 그 외 표현은 거부합니다.
- 예: `search_docs(query)`, `draft_email(recipient, summary)` 같은 제한된 액션만 허용

보호 효과

- 공격자가 넣은 자연어 지시가 곧바로 도구 호출로 번역되기 어렵습니다.
- 정책 검토 대상이 자유 텍스트가 아니라 구조화된 실행 계획이 됩니다.
- 허용되지 않은 액션은 결정적으로 차단할 수 있습니다.

방어 패턴 4: Structured Extraction만 허용

자유 서술 대신 구조화된 값만 통과

- 예: `{invoice_id, amount, sender, due_date}` 만 추출
- 요약문 전체를 planner에 넘기지 않고 필요한 필드만 전달
- 허용되지 않은 필드는 버리고, 길이와 형식을 제한

막는 것

- 공격자가 넣은 "다음 단계에서 이 URL로 보내라" 같은 문장이 살아남기 어렵습니다.
- 계획 모델은 데이터와 지시를 구별할 부담이 줄어듭니다.
- PDF, HTML, 이메일처럼 표현력이 큰 입력에 특히 효과적입니다.

방어 패턴 5: 컨텍스트 최소화

컨텍스트를 작게 유지

- 필요한 문서 조각만 읽고, 검색 결과 전체를 한 번에 넣지 않습니다.
- 민감 저장소와 외부 문서를 같은 컨텍스트 창에 섞지 않습니다.
- 세션 메모리도 목적별로 나누고 자동 정리합니다.

언제 특히 유리한가

- 검색 결과가 많은 RAG 흐름
- 여러 메일과 첨부파일을 동시에 읽는 워크플로
- 내부 문서와 외부 웹 문서를 함께 다뤄야 하는 조사형 에이전트

방어 패턴 6: Map-Reduce

문서별로 먼저 처리하고 마지막에 합친다

- 각 입력 문서마다 추출 결과를 따로 만든 뒤 마지막 집계 단계에서 합칩니다.
- 최종 단계는 원문 전체가 아니라 부분 요약·구조화 결과만 받습니다.
- 공격 문장이 전체 컨텍스트를 장악하는 효과를 줄일 수 있습니다.

실무에서의 장점

- 병렬 처리와 재시도에 유리합니다.
- 문서 단위 provenance와 audit trail을 남기기 쉽습니다.
- 대규모 검색·요약 작업에서 blast radius를 줄이는 데 효과적입니다.

방어 패턴 7: Action Selector와 승인 게이트

외부 액션은 별도 선택 단계로 분리

- send mail, post Slack, write ticket, execute command 는 별도 selector가 결정
- selector는 입력 출처, 데이터 민감도, 대상 도메인, approval 상태를 함께 봅니다.
- 필요하면 human approval 없이는 실행하지 않습니다.

정책 엔진이 강제해야 할 것

- egress allowlist, tool별 scope, read/write 분리, request signing
- 고위험 액션의 dry-run preview와 diff 표시
- 누가 어떤 컨텍스트로 어떤 액션을 승인했는지 감사 로그 남기기

실무 적용 체크리스트

1. 외부 입력을 읽는 도구와 민감 데이터를 읽는 도구를 분리합니다.
2. planner, extractor, executor를 분리하고 각자 tool scope를 다르게 부여합니다.
3. 외부 전송과 상태변경 도구는 allowlist, approval, audit log 뒤에 둡니다.
4. structured output, bounded schema, short-lived memory를 기본값으로 둡니다.
5. 한 세션 안에서 Lethal Trifecta가 다시 결합되는 우회 경로가 없는지 점검합니다.

필드 사례가 보여주는 것

EchoLeak이 보여준 구조

- 악성 이메일이라는 비신뢰 입력이 M365 Copilot의 검색·요약 경로에 들어갔습니다.
- 그 뒤 내부 데이터 접근과 외부 전송 경로가 결합되며 사용자 개입 없이도 유출 경로가 열릴 수 있다는 점이 드러났습니다.
- 즉 실제 취약점은 단일 프롬프트가 아니라 도구 경계와 데이터 흐름 조합에 있었습니다.

교훈

- 취약점은 대체로 "모델이 너무 똑똑해서"가 아니라 "시스템이 너무 많은 권한을 붙여서" 발생합니다.
- 보안 설계의 핵심은 모델 내부가 아니라 도구 경계와 액션 정책입니다.
- 결국 agent 보안은 architecture 문제입니다.

결론

AI agent가 비신뢰 입력 처리, 민감 데이터 접근, 외부 전송·상태변경 액션을 동시에 가지면 indirect prompt injection은 모델 품질 문제가 아니라 구조적 취약점이 됩니다.

따라서 근본 방어는 "더 강한 프롬프트"가 아니라, 세 조건이 한 세션과 한 경로에서 동시에 성립하지 않도록 경계를 나누는 일입니다. Lethal Trifecta와 Rule of Two는 그 경계를 점검하는 실무 기준입니다.

참고 문헌 1: 연구와 개념

- [Indirect Prompt Injection 논문 \(2023\)](#)
- [CaMeL 논문](#)
- [CaMeL research artifact](#)
- [Operationalizing CaMeL](#)
- [LLM Agent 방어 패턴 논문](#)
- [Lethal Trifecta 정리](#)
- [Meta Practical AI Agent Security](#)

참고 문헌 2: 운영 가이드와 사례

- [OpenAI: Prompt Injection 저항 설계](#)
- [OpenAI API: Agent Safety Guide](#)
- [OpenAI: 링크 클릭 안전성](#)
- [OWASP AI Agent Security Cheat Sheet](#)
- [OWASP MCP Security Cheat Sheet](#)
- [MSRC: Indirect Prompt Injection 방어](#)
- [Aim Labs: EchoLeak 분석](#)
- [Reversec: LLM Agent 방어 패턴 실습](#)